Algoritmi ricorsivi e iterativi a confronto

Indice

Ricorsione e iterazione

Confronto in sintesi

Il divide et impera dà il guadagno, non la ricorsione in sé

Quando i sottoproblemi si sovrappongono

Spazio

Esperimento

Linee guida

Dove eravamo

Nei capitoli precedenti abbiamo studiato diversi algoritmi ricorsivi:

- capitolo «Ricorsione»: fattoriale, numeri di Fibonacci, MCD ricorsivo, potenza veloce, Torre di Hanoi;
- capitolo «Algoritmi ricorsivi»: somma di un array, Merge Sort, ricerca binaria ricorsiva, sotto-array di somma massima.

Li abbiamo analizzati con le equazioni di ricorrenza e con il Master theorem.

La domanda centrale

In questo capitolo rispondiamo a una domanda molto pratica:

quando conviene progettare un algoritmo ricorsivo,
e quando no?

Dipende dalla scomposizione

La risposta non dipende dalla ricorsione in sé, ma da **come** il problema viene scomposto in sottoproblemi:

- se il problema viene suddiviso in pochi sottoproblemi **indipendenti** di dimensione n/b (divide et impera), la ricorsione può dare un grande guadagno asintotico;
- se il problema viene solo ridotto dalla dimensione n alla dimensione $n-1$, la ricorsione non dà alcun guadagno in tempo e costa memoria;
- se i sottoproblemi si **sovrappongono**, un algoritmo ricorsivo ingenuo può essere esponenzialmente più lento di uno iterativo.

Piano del capitolo

- Innanzitutto, i costi che ogni chiamata ricorsiva comporta
- Poi un confronto tra algoritmi ricorsivi e iterativi per i problemi del corso
- Le tre situazioni precedenti, una a una: divide et impera, decremento di uno (con la memoria usata dalla ricorsione), sottoproblemi sovrapposti
- Infine, i tempi di esecuzione misurati delle implementazioni in C++ e alcune linee guida

Ricorsione e iterazione

Introduzione: Ricorsione e iterazione

- In questa sezione vediamo che ricorsione e iterazione possono esprimere esattamente gli stessi algoritmi
- Guardiamo che cosa succede nel calcolatore quando una funzione viene chiamata: la **pila delle chiamate**, e i tre costi nascosti della ricorsione
- Infine, trasformiamo un algoritmo ricorsivo in uno iterativo con una pila esplicita, e vediamo che cosa cambia e che cosa no

Stessa potenza espressiva

- Ogni algoritmo ricorsivo può essere trasformato in uno iterativo (che usa una **pila esplicita** per memorizzare le chiamate in sospeso)
- Ogni algoritmo iterativo può essere scritto in modo ricorsivo (ogni ciclo può essere sostituito da una funzione che chiama se stessa)

Ricorsione e iterazione hanno la **stessa potenza espressiva**:
ciò che cambia dall'una all'altra è il **numero di passi**
e la **quantità di memoria** necessari per risolvere il problema.

La pila delle chiamate

- Ogni volta che una funzione viene chiamata, il calcolatore crea un **frame** (o *record di attivazione*) nella **pila delle chiamate** (*call stack*), un'area di memoria riservata al programma

- Il frame contiene:
  - i parametri della chiamata;
  - le variabili locali della chiamata;
  - l'indirizzo di ritorno, cioè il punto della funzione chiamante da cui riprende l'esecuzione quando la chiamata termina.

Le chiamate attive mantengono il loro frame

- Il frame viene rimosso dalla pila solo quando la chiamata termina
- Di conseguenza, durante l'esecuzione di un algoritmo ricorsivo, tutte le chiamate che sono iniziate e non ancora terminate mantengono il loro frame nella pila
- Questo produce tre **costi nascosti**, che non compaiono in un ciclo

I costi nascosti della ricorsione (1/2)

1. **Sovraccarico delle chiamate**: creare e rimuovere un frame, copiare i parametri e saltare alla funzione e tornare indietro richiedono un numero costante di passi aggiuntivi per ogni chiamata.
   - Questo non cambia l'ordine di crescita del tempo di esecuzione
   - ma rende un algoritmo ricorsivo più lento di un ciclo equivalente di un fattore costante

2. **Memoria**: la memoria usata dalla pila delle chiamate è proporzionale al massimo numero di chiamate *contemporaneamente* attive, cioè alla **profondità** della ricorsione (l'altezza dell'albero di ricorsione più uno, studiata in dettaglio più avanti in questo capitolo).

I costi nascosti della ricorsione (2/2)

3. **Stack overflow**: la pila delle chiamate ha una dimensione fissa e piuttosto piccola.
   - Su Linux la dimensione predefinita è tipicamente 8 MB (comando `ulimit -s` $\Rightarrow$ 8192 KB), su Windows è tipicamente 1 MB
   - Se la profondità della ricorsione è troppo grande, la pila si esaurisce e il programma va in **crash** (in C++ con un *segmentation fault*)
   - In Python il limite predefinito è di sole 1000 chiamate annidate

Esempio (1/2)

Una somma lineare ricorsiva che va in crash.

- Consideriamo l'algoritmo ricorsivo per decremento di uno che calcola la somma di un array $a[1..n]$ come

$$\texttt{SumDecrease}(a, n) = \texttt{SumDecrease}(a, n-1) + a[n],$$

  con caso base $\texttt{SumDecrease}(a, 0) = 0$ (la somma per decremento di uno del capitolo «Algoritmi ricorsivi», con caso base $n = 0$ invece di $n = 1$)

- Le chiamate $\texttt{SumDecrease}(a, n), \texttt{SumDecrease}(a, n-1), \ldots, \texttt{SumDecrease}(a, 0)$ sono tutte attive contemporaneamente: la profondità della ricorsione è $n + 1$

Esempio (2/2)

- Con $n = 10^7$ e un frame di circa 32 byte, la pila dovrebbe contenere circa

$$10^7 \cdot 32 = 3.2 \cdot 10^8 \text{ byte} \approx 320 \text{ MB},$$

  cioè 40 volte gli 8 MB predefiniti

- Il programma va in crash, benché l'array stesso ($4 \cdot 10^7$ byte $= 40$ MB) stia comodamente in memoria e la somma iterativa calcoli il risultato in circa 3 millisecondi

- Nell'esperimento alla fine del capitolo misuriamo che, con una pila di 8 MB, il crash avviene già per $n \approx 7 \cdot 10^5$

Dalla ricorsione all'iterazione con una pila esplicita

- Un algoritmo ricorsivo può sempre essere trasformato in uno iterativo gestendo **esplicitamente** una pila che contiene i parametri delle chiamate ancora da eseguire
- Come esempio, scriviamo una versione iterativa della somma ricorsiva per dimezzamento `RecursiveSum` del capitolo «Algoritmi ricorsivi»
- `RecursiveSum` calcola la somma di $a[l..r]$ come la somma di $a[l..mid]$ più la somma di $a[mid + 1..r]$

Questo è lo pseudocodice dell'algoritmo `SumExplicitStack`,
che calcola la somma di un array $a[1..n]$
tenendo in una pila P i sotto-array ancora da sommare.

Algorithm: SumExplicitStack (1/2)

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output : La somma $a[1] + a[2] + \cdots + a[n]$

```
1 function SumExplicitStack(a, n)
2 |   s := 0
3 |   P := pila vuota                    // sotto-array in sospeso
4 |   inserisci (1, n) in P
5 |   while P non è vuota do
6 |   |   (l, r) := estrai da P
```

Algorithm: SumExplicitStack (2/2)

```
function SumExplicitStack(a, n) (continua)
    while P non è vuota (continua) do
 7      if l = r then
 8          s ← s + a[l]                          // caso base
 9      else
10          mid := ⌊(l + r)/2⌋
11          inserisci (mid + 1, r) in P
12          inserisci (l, mid) in P
13      end
14  end
15  return s
16 end
```

Esegui in Colab: Python · C++ · Java

Il ruolo della pila P

- La pila P svolge esattamente il ruolo della pila delle chiamate: contiene i sotto-array ancora da sommare
- La sua dimensione massima è dello stesso ordine della profondità della ricorsione, cioè

$$\Theta(\log_2 n)$$

Esempio (1/2)

Eseguiamo `SumExplicitStack`$(a, 4)$ con $a = [4, 7, 1, 9]$. Ogni riga riporta la coppia estratta da P, ciò che viene fatto, e il contenuto di P dopo (la cima della pila è a destra):

estrai	azione	P dopo	s
$(1, 4)$	$mid = 2$, inserisci $(3, 4)$ e $(1, 2)$	$[(3, 4), (1, 2)]$	0
$(1, 2)$	$mid = 1$, inserisci $(2, 2)$ e $(1, 1)$	$[(3, 4), (2, 2), (1, 1)]$	0
$(1, 1)$	caso base	$[(3, 4), (2, 2)]$	4
$(2, 2)$	caso base	$[(3, 4)]$	11
$(3, 4)$	$mid = 3$, inserisci $(4, 4)$ e $(3, 3)$	$[(4, 4), (3, 3)]$	11
$(3, 3)$	caso base	$[(4, 4)]$	12
$(4, 4)$	caso base	vuota	21

Esempio (2/2)

- L'algoritmo restituisce

$$21 = 4 + 7 + 1 + 9$$

- I sotto-array vengono elaborati nello stesso ordine delle chiamate di `RecursiveSum`$(a, 1, 4)$
- P non contiene mai più di 3 coppie

Che cosa cambia con la pila esplicita

- Simulare la ricorsione con una pila esplicita **non** cambia l'ordine di crescita del numero di passi e della memoria

- Elimina solo il sovraccarico delle chiamate e sposta la pila dalla (piccola) pila delle chiamate alla (grande) memoria principale

Un vero guadagno in tempo o in memoria può venire solo da un **algoritmo diverso**,
non dalla sostituzione meccanica della ricorsione con l'iterazione (o viceversa).

Riepilogo

Ricorsione e iterazione:

- stessa potenza espressiva: cambiano il numero di passi e la memoria

Costi nascosti della ricorsione:

- sovraccarico delle chiamate (un fattore costante), memoria proporzionale alla profondità della ricorsione, rischio di stack overflow

Pila esplicita:

- `SumExplicitStack` simula `RecursiveSum` con una pila di $\Theta(\log_2 n)$ coppie
- stesso ordine di crescita: un vero guadagno richiede un algoritmo diverso

Confronto in sintesi

Introduzione: Confronto in sintesi

- In questa sezione confrontiamo, per i problemi studiati nel corso, il miglior algoritmo iterativo del corso con quello ricorsivo
- Il confronto dà una mappa del capitolo: i casi in cui la ricorsione conviene, quelli in cui sostituisce soltanto un ciclo, e il caso in cui è un disastro
- Ciascuno di questi casi viene studiato in dettaglio nelle sezioni successive

Come leggere il confronto

- Il **tempo** è il numero di passi $S(n)$ nel caso peggiore (le operazioni aritmetiche sui numeri contano come un passo, come nel capitolo «Tempo di esecuzione»)
- Lo **spazio** è la memoria usata **in aggiunta all'input**, compresa la pila delle chiamate
- L'esito si riferisce all'algoritmo ricorsivo:
  - **GUADAGNO** se è asintoticamente più veloce;
  - **UGUALE** se ha lo stesso tempo di esecuzione;
  - **PERDITA** se è peggiore in tempo o in spazio.
- $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ è la sezione aurea e $m = \min(a, b)$ per l'MCD di a e b

La ricorsione conviene (1/2)

Problema	**Iterativo**	**tempo**	**spazio**	**Ricorsivo**	**tempo**	**spazio**
Ordinamento	Insertion, Selection, Exchange Sort	$\Theta(n^2)$	$\Theta(1)$	Merge Sort	$\Theta(n \log_2 n)$	$\Theta(n)$
Potenza x^n	n moltiplicazioni	$\Theta(n)$	$\Theta(1)$	Potenza veloce	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$
Sotto-array di somma massima	Forza bruta	$\Theta(n^2)$	$\Theta(1)$	Divide et impera	$\Theta(n \log_2 n)$	$\Theta(\log_2 n)$
Prodotto di interi di n cifre	Metodo scolastico	$\Theta(n^2)$	$\Theta(n)$	Karatsuba	$\Theta(n^{\log_2 3})$	$\Theta(n)$

La ricorsione conviene (2/2)

Problema	**Esito**
Ordinamento	**GUADAGNO** in tempo (divide et impera), pagato con $\Theta(n)$ di memoria
Potenza x^n	**GUADAGNO**, che deriva dal dimezzamento di n: anche l'algoritmo iterativo square-and-multiply è $\Theta(\log_2 n)$, con spazio $\Theta(1)$
Sotto-array di somma massima	**GUADAGNO** rispetto alla forza bruta, ma l'algoritmo iterativo di Kadane, con tempo $\Theta(n)$ e spazio $\Theta(1)$, batte entrambi
Prodotto di interi di n cifre	**GUADAGNO**: $S(n) = 3S(n/2) + \Theta(n)$, $n^{\log_2 3} \approx n^{1.585}$

La ricorsione sostituisce soltanto un ciclo (1/2)

Problema	**Iterativo**	**tempo**	**spazio**	**Ricorsivo**	**tempo**	**spazio**
Somma di un array	Ciclo	$\Theta(n)$	$\Theta(1)$	Dimezzamento Decr. di uno	$\Theta(n)$ $\Theta(n)$	$\Theta(\log_2 n)$ $\Theta(n)$
Ricerca binaria	Ciclo	$\Theta(\log_2 n)$	$\Theta(1)$	Ricerca binaria ricorsiva	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$
Fattoriale, ricerca lineare	Ciclo	$\Theta(n)$	$\Theta(1)$	Decremento di uno	$\Theta(n)$	$\Theta(n)$
MCD (Euclide)	Ciclo	$\Theta(\log m)$	$\Theta(1)$	MCD ricorsivo	$\Theta(\log m)$	$\Theta(\log m)$

La ricorsione sostituisce soltanto un ciclo (2/2)

Problema	**Esito**
Somma di un array	**UGUALE** tempo, **PERDITA** in spazio
Ricerca binaria	**UGUALE** tempo, **PERDITA** in spazio; ricorsione in coda, si può eliminare
Fattoriale, ricerca lineare	**UGUALE** tempo, **PERDITA** in spazio, rischio di stack overflow
MCD (Euclide)	**UGUALE**: ricorsione in coda, la profondità è solo logaritmica

Sottoproblemi sovrapposti e Torre di Hanoi

Problema	**Iterativo**	**tempo**	**spazio**	**Ricorsivo**	**tempo**	**spazio**
Numeri di Fibonacci	Ciclo con due variabili	$\Theta(n)$	$\Theta(1)$	Ricorsione ingenua	$\Theta(\varphi^n)$	$\Theta(n)$
Torre di Hanoi	Esistono versioni iterative	$\Theta(2^n)$	$\Theta(n)$	Hanoi ricorsivo	$\Theta(2^n)$	$\Theta(n)$

Esito:

- Numeri di Fibonacci: enorme **PERDITA**: sottoproblemi sovrapposti; la memoizzazione lo riporta a $\Theta(n)$
- Torre di Hanoi: **UGUALE**: servono comunque $2^n - 1$ mosse; la ricorsione è la descrizione naturale

Tre blocchi, tre situazioni

- Le quattro righe del primo blocco sono i casi in cui la ricorsione **conviene**: in tutti il problema viene suddiviso in sottoproblemi di dimensione $n/2$ (sezione successiva)
- Le righe del secondo blocco sono i casi in cui la ricorsione **sostituisce** soltanto un ciclo: nessun guadagno in tempo, una perdita in memoria (sezione «Spazio»)
- La riga di Fibonacci è il caso in cui la ricorsione ingenua è un **disastro**, perché gli stessi sottoproblemi vengono risolti più e più volte (sezione «Quando i sottoproblemi si sovrappongono»)

Riepilogo

La ricorsione conviene:

- ordinamento, potenza, sotto-array di somma massima, prodotto di interi: sottoproblemi di dimensione $n/2$

La ricorsione sostituisce soltanto un ciclo:

- somma di un array, ricerca binaria, fattoriale, ricerca lineare, MCD: stesso tempo, più memoria

La ricorsione è un disastro:

- numeri di Fibonacci con la ricorsione ingenua: $\Theta(\varphi^n)$ invece di $\Theta(n)$

Il divide et impera dà il guadagno, non la ricorsione in sé

Introduzione: Divide et impera

- In questa sezione confrontiamo le due famiglie di equazioni di ricorrenza del corso: decremento di uno e divide et impera
- Mostriamo che una ricorsione per decremento di uno svolge lo stesso lavoro di un ciclo, mentre il divide et impera può dare un guadagno asintotico
- Per il Merge Sort e la potenza veloce scriviamo algoritmi iterativi con lo stesso tempo di esecuzione: il guadagno viene dalla scomposizione, non dalla ricorsione
- Incontriamo anche un algoritmo iterativo che batte il divide et impera (Kadane)

Due famiglie di equazioni di ricorrenza

Il numero di passi di un algoritmo ricorsivo è descritto da un'equazione di ricorrenza. Nel corso compaiono due famiglie di equazioni di ricorrenza:

- **decremento di uno**: il problema di dimensione n viene ridotto a **un** problema di dimensione $n-1$,

$$S(n) = S(n-1) + f(n);$$

- **divide et impera**: il problema di dimensione n viene suddiviso in a problemi di dimensione n/b,

$$S(n) = a\,S(n/b) + f(n), \qquad a \geq 1,\ b > 1.$$

Decremento di uno: la ricorsione è un ciclo sotto mentite spoglie

Osservazione 1: equazioni di ricorrenza per decremento di uno

Sia $S(n) = S(n-1) + f(n)$ *per* $n \geq 1$, *con* $S(0) = d$. *Allora*

$$S(n) = d + \sum_{i=1}^{n} f(i).$$

In particolare, se $f(n) = c$ *per una costante* $c > 0$, *allora* $S(n) = c\,n + d = \Theta(n)$.

Dimostrazione

Per induzione su n. Per $n = 0$ la somma è vuota e $S(0) = d$.

Se la formula vale per $n - 1$, allora

$$S(n) = S(n-1) + f(n)$$

$$= d + \sum_{i=1}^{n-1} f(i) + f(n)$$

$$= d + \sum_{i=1}^{n} f(i).$$

Se $f(i) = c$ per ogni i, allora $\sum_{i=1}^{n} c = c\,n$ e $S(n) = c\,n + d$. Poiché

$$\lim_{n\to\infty} \frac{c\,n + d}{n} = c \in \mathbb{R}_{>0},$$

si ha $S(n) = \Theta(n)$. □

Un ciclo sotto mentite spoglie (1/2)

- La formula

$$S(n) = d + \sum_{i=1}^{n} f(i)$$

è **esattamente** il numero di passi di un ciclo che esegue, per $i = 1, 2, \ldots, n$, un corpo di costo $f(i)$

- La ricorsione per decremento di uno svolge lo stesso lavoro di un ciclo

Un ciclo sotto mentite spoglie (2/2)

- Fattoriale, ricerca lineare e somma di un array per decremento di uno:

$$S(n) = S(n-1) + \Theta(1) = \Theta(n),$$

  come il ciclo corrispondente
- Un insertion sort ricorsivo (ordinare $a[1..n-1]$ ricorsivamente, poi inserire $a[n]$):

$$S(n) = S(n-1) + \Theta(n) = \Theta(n^2),$$

  come l'`InsertionSort` iterativo
- In tutti questi casi l'algoritmo ricorsivo ha lo stesso ordine di crescita del tempo di esecuzione di quello iterativo, più il sovraccarico delle chiamate e una profondità di ricorsione $\Theta(n)$

Divide et impera: da dove viene il guadagno

- Il Merge Sort suddivide l'array in due metà, le ordina ricorsivamente e le fonde in $\Theta(n)$ passi:

$$S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n)$$

  per il caso 2 del Master theorem

- Il guadagno rispetto a $\Theta(n^2)$ deriva dal fatto che le due metà sono **bilanciate** e **indipendenti**: l'albero di ricorsione ha solo $\Theta(\log_2 n)$ livelli, e ogni livello costa $\Theta(n)$

Merge Sort bottom-up

- La stessa suddivisione si può organizzare **senza ricorsione**:
  - si fondono prima coppie di sotto-array di lunghezza 1,
  - poi coppie di sotto-array di lunghezza 2,
  - poi di lunghezza 4, e così via.

- Questo è il Merge Sort **bottom-up**

- Usa la procedura `Merge`(a, l, m, r) del capitolo «Algoritmi ricorsivi», che fonde i sotto-array ordinati $a[l..m]$ e $a[m+1..r]$ in $\Theta(r - l + 1)$ passi

Questo è lo pseudocodice dell'algoritmo `MergeSortBottomUp`,
che ordina un array $a[1..n]$
fondendo sotto-array di lunghezza $1, 2, 4, \ldots$

Algorithm: MergeSortBottomUp (1/2)

Input : Un array $a[1..n]$ di n numeri naturali
Output : L'array a ordinato in ordine non decrescente

```
1 function MergeSortBottomUp(a, n)
2 |   width := 1
3 |   while width < n do
4 |   |   l := 1
5 |   |   while l + width ≤ n do
6 |   |   |   m := l + width − 1
```

Algorithm: MergeSortBottomUp (2/2)

```
function MergeSortBottomUp(a, n) (continua)
    while width < n (continua) do
        while l + width ≤ n (continua) do
7           r := min(l + 2 · width − 1, n)
8           Merge(a, l, m, r)
9           l ← l + 2 · width
10      end
11      width ← 2 · width
12  end
13 end
```

Esegui in Colab: Python · C++ · Java

Esempio

Consideriamo $a = [5, 2, 4, 6, 1, 3]$ ($n = 6$).

- $width = 1$: si fondono $a[1..1]$ con $a[2..2]$, $a[3..3]$ con $a[4..4]$, $a[5..5]$ con $a[6..6] \Rightarrow a = [2, 5, 4, 6, 1, 3]$
- $width = 2$: si fondono $a[1..2]$ con $a[3..4]$; per $l = 5$ si ha $l + width = 7 > 6$, quindi $a[5..6]$ resta com'è $\Rightarrow$ $a = [2, 4, 5, 6, 1, 3]$
- $width = 4$: si fondono $a[1..4]$ con $a[5..6] \Rightarrow a = [1, 2, 3, 4, 5, 6]$
- $width = 8 \geq 6$: ci si ferma

L'algoritmo esegue $3 = \lceil \log_2 6 \rceil$ passate.

Osservazione 2: Merge Sort bottom-up

Per $n \geq 2$, `MergeSortBottomUp` esegue $\lceil \log_2 n \rceil$ passate del ciclo esterno e il suo numero di passi nel caso peggiore è

$$S(n) = \Theta(n \log_2 n),$$

lo stesso del Merge Sort ricorsivo. Inoltre, non usa la pila delle chiamate.

- Idea della dimostrazione: limitiamo il costo di una passata dall'alto e dal basso con un multiplo di n, e contiamo le passate

Dimostrazione (1/3)

Sia $n \geq 2$.

Nel ciclo esterno *width* assume i valori

$$1, 2, 4, \ldots, 2^{p-1},$$

dove p è il più piccolo intero con $2^p \geq n$, cioè

$$p = \lceil \log_2 n \rceil.$$

Dimostrazione (2/3)

Limite superiore.

- In una passata, le chiamate a `Merge` (riga 8) lavorano su sotto-array disgiunti, quindi la loro lunghezza totale è al più n e il loro costo totale è $O(n)$
- Il ciclo interno viene eseguito al più $\frac{n}{2 \cdot width} + 1 \leq n$ volte, ogni volta con un numero costante di altri passi
- Quindi ogni passata costa $O(n)$ e

$$S(n) = O(n \lceil \log_2 n \rceil) = O(n \log_2 n).$$

Dimostrazione (3/3)

Limite inferiore.

- Il ciclo interno si ferma quando $l + \mathit{width} > n$, quindi gli elementi che non vengono fusi in una passata sono al più width
- Nelle passate con $\mathit{width} \leq n/2$, vengono fusi almeno $n - \mathit{width} \geq n/2$ elementi, con $\Omega(n)$ passi
- Queste passate sono quelle con $\mathit{width} = 2^t$, $t = 0, 1, \ldots, \lfloor \log_2 n \rfloor - 1$, cioè $\lfloor \log_2 n \rfloor$ passate. Quindi $S(n) = \Omega(n \lfloor \log_2 n \rfloor) = \Omega(n \log_2 n)$

Quindi $S(n) = \Theta(n \log_2 n)$. L'algoritmo non contiene chiamate ricorsive, quindi l'unica memoria aggiuntiva è quella usata da `Merge`, cioè $\Theta(n)$. □

Il guadagno viene dal divide et impera

Il guadagno del Merge Sort rispetto
agli algoritmi di ordinamento $\Theta(n^2)$
è una conseguenza della strategia **divide et impera**
(metà bilanciate, fuse in tempo lineare),
non della ricorsione.

La versione iterativa bottom-up
ha lo stesso tempo di esecuzione $\Theta(n \log_2 n)$.

Potenza veloce: il guadagno viene dal dimezzamento

- La potenza veloce ricorsiva del capitolo «Ricorsione» calcola

$$x^n = \begin{cases} \left(x^{\lfloor n/2 \rfloor}\right)^2 & \text{se } n \text{ è pari,} \\ \left(x^{\lfloor n/2 \rfloor}\right)^2 \cdot x & \text{se } n \text{ è dispari,} \end{cases}$$

  con

$$S(n) = S(\lfloor n/2 \rfloor) + \Theta(1) = \Theta(\log_2 n)$$

- Il dimezzamento dell'esponente si può realizzare anche con un ciclo, leggendo i bit di n dal meno significativo (**square-and-multiply**)

Algorithm: PowerIt (1/2)

Input : Un numero x e un numero naturale n
Output : La potenza x^n

```
1 function PowerIt(x, n)
2 |   p := 1
3 |   y := x
4 |   k := n
5 |   while k > 0 do
6 |   |   if k è dispari then
7 |   |   |   p ← p · y
8 |   |   end
```

Algorithm: PowerIt (2/2)

```
function PowerIt(x, n) (continua)
    while k > 0 (continua) do
9       y ← y · y
10      k ← ⌊k/2⌋
11  end
12  return p
13 end
```

Esegui in Colab: Python · C++ · Java

Perché PowerIt è corretto

- All'inizio di ogni iterazione si ha

$$p \cdot y^k = x^n$$

- Vale all'inizio con $p = 1$, $y = x$, $k = n$, ed è preservata da ogni iterazione

- Quando $k = 0$ si ottiene $p = x^n$

Esempio

Eseguiamo `PowerIt`$(x, 13)$, dove $13 = (1101)_2$. Ogni elemento riporta i valori alla fine di un'iterazione:

- $k = 13$ è dispari: $p = x$, $y = x^2$, $k = 6$;
- $k = 6$ è pari: $p = x$, $y = x^4$, $k = 3$;
- $k = 3$ è dispari: $p = x \cdot x^4 = x^5$, $y = x^8$, $k = 1$;
- $k = 1$ è dispari: $p = x^5 \cdot x^8 = x^{13}$, $y = x^{16}$, $k = 0$.

Le iterazioni moltiplicano p per x, x^4 e x^8 (i bit di 13 uguali a 1), e

$$p = x^{1+4+8} = x^{13}$$

dopo $4 = \lfloor \log_2 13 \rfloor + 1$ iterazioni.

Osservazione 3: iterazioni di PowerIt

Per $n \geq 1$, `PowerIt` esegue esattamente $\lfloor \log_2 n \rfloor + 1$ iterazioni, quindi $S(n) = \Theta(\log_2 n)$, e usa $\Theta(1)$ di memoria aggiuntiva.

Dimostrazione

- Ogni iterazione sostituisce k con $\lfloor k/2 \rfloor$, cioè elimina il bit meno significativo della rappresentazione binaria di k
- Quindi il numero di iterazioni è il numero di cifre binarie di n, che è

$$\lfloor \log_2 n \rfloor + 1$$

(capitolo «Rappresentazione dei numeri»)

- Ogni iterazione esegue un numero costante di passi, quindi $S(n) = \Theta(\log_2 n)$
- Vengono usate solo le tre variabili p, y, k

□

Sotto-array di somma massima: un algoritmo iterativo migliore

- Per il problema del sotto-array di somma massima (capitolo «Algoritmi ricorsivi») l'algoritmo divide et impera, con

$$S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n),$$

  migliora l'algoritmo di forza bruta $\Theta(n^2)$
- Tuttavia, un algoritmo **iterativo** più astuto, dovuto a Kadane, risolve il problema in $\Theta(n)$ passi
- Per ogni posizione i mantiene la somma massima *cur* di un sotto-array che **termina** in posizione i: tale sotto-array o è il solo $a[i]$, oppure estende il miglior sotto-array che termina in $i - 1$

Questa è una versione compatta dell'algoritmo `Kadane`
del capitolo «Algoritmi ricorsivi»,
che restituisce solo la somma massima.

Algorithm: Kadane

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output : La somma massima di un sotto-array non vuoto $a[l..r]$

```
1  function Kadane(a, n)
2      best := a[1]
3      cur := a[1]  // miglior somma di un sotto-array che
        termina in i
4      i := 2
5      while i ≤ n do
6          cur ← max(a[i], cur + a[i])
7          best ← max(best, cur)
8          i ← i + 1
9      end
10     return best
11 end
```

Esegui in Colab: Python · C++ · Java

Kadane batte il divide et impera

- Il ciclo viene eseguito $n - 1$ volte con un numero costante di passi:

$$S(n) = \Theta(n),$$

  con $\Theta(1)$ di memoria aggiuntiva

- Il divide et impera è una potente **tecnica di progetto**, ma non è sempre la migliore

- Un guadagno rispetto alla forza bruta non significa che l'algoritmo ricorsivo sia ottimo

Moltiplicazione di interi: Karatsuba

- Due interi di n cifre si moltiplicano con il metodo scolastico in $\Theta(n^2)$ passi sulle cifre
- Suddividendo ciascun numero in due metà di $n/2$ cifre, il prodotto richiede quattro prodotti di numeri di $n/2$ cifre
- Karatsuba ha osservato che bastano **tre** prodotti (più addizioni e scorrimenti, che costano $\Theta(n)$):

$$S(n) = 3S(n/2) + \Theta(n)$$

- Qui $a = 3$, $b = 2$, $f(n) = \Theta(n)$ e $n^{\log_2 3} \approx n^{1.585}$; poiché $f(n) = O(n^{\log_2 3 - \varepsilon})$ per $\varepsilon = 0.5$, il caso 1 del Master theorem dà $S(n) = \Theta(n^{\log_2 3})$, asintoticamente migliore di $\Theta(n^2)$

Punti chiave

- Una ricorsione per decremento di uno svolge lo stesso lavoro di un ciclo
- Una ricorsione divide et impera può essere asintoticamente più veloce dei semplici algoritmi iterativi (Merge Sort, potenza veloce, Karatsuba)
- Ma il guadagno deriva dalla scomposizione in sottoproblemi indipendenti e bilanciati, e la stessa scomposizione si può spesso organizzare con un ciclo (Merge Sort bottom-up, square-and-multiply)
- Il miglior algoritmo per un problema può addirittura essere iterativo (Kadane)

Riepilogo (1/2)

Decremento di uno:

- $S(n) = S(n-1) + f(n)$ dà $S(n) = d + \sum_{i=1}^{n} f(i)$: i passi di un ciclo
- stesso tempo di esecuzione del ciclo, più il sovraccarico delle chiamate e profondità $\Theta(n)$

Divide et impera:

- metà bilanciate e indipendenti: Merge Sort in $\Theta(n \log_2 n)$
- `MergeSortBottomUp`: lo stesso $\Theta(n \log_2 n)$ senza ricorsione

Riepilogo (2/2)

Potenza veloce:

- `PowerIt` (square-and-multiply): $\lfloor \log_2 n \rfloor + 1$ iterazioni, memoria $\Theta(1)$

Sotto-array di somma massima:

- il `Kadane` iterativo in $\Theta(n)$ batte il divide et impera in $\Theta(n \log_2 n)$

Karatsuba:

- $S(n) = 3S(n/2) + \Theta(n) = \Theta(n^{\log_2 3})$, migliore di $\Theta(n^2)$

Quando i sottoproblemi si sovrappongono

Introduzione: Quando i sottoproblemi si sovrappongono

- In questa sezione studiamo il caso in cui i sottoproblemi non sono disgiunti
- Sui numeri di Fibonacci vediamo che la ricorsione ingenua esegue un numero esponenziale di chiamate
- Poi eliminiamo lo spreco in due modi: la ricorsione con memoizzazione, e un semplice ciclo
- È l'idea della programmazione dinamica, che verrà studiata più avanti nel corso

Sottoproblemi disgiunti e sovrapposti

- Nel divide et impera i sottoproblemi sono **disgiunti**: le due metà del Merge Sort non hanno alcun elemento in comune
- Quando i sottoproblemi si **sovrappongono**, un algoritmo ricorsivo ingenuo risolve molte volte lo stesso sottoproblema
- I numeri di Fibonacci sono definiti da

$$F(0) = 0, \qquad F(1) = 1,$$

$$F(n) = F(n-1) + F(n-2) \quad \text{per } n \geq 2$$

- La traduzione diretta della definizione è l'algoritmo `FibRec`

Algorithm: FibRec

Input : Un numero naturale n
Output : Il numero di Fibonacci $F(n)$

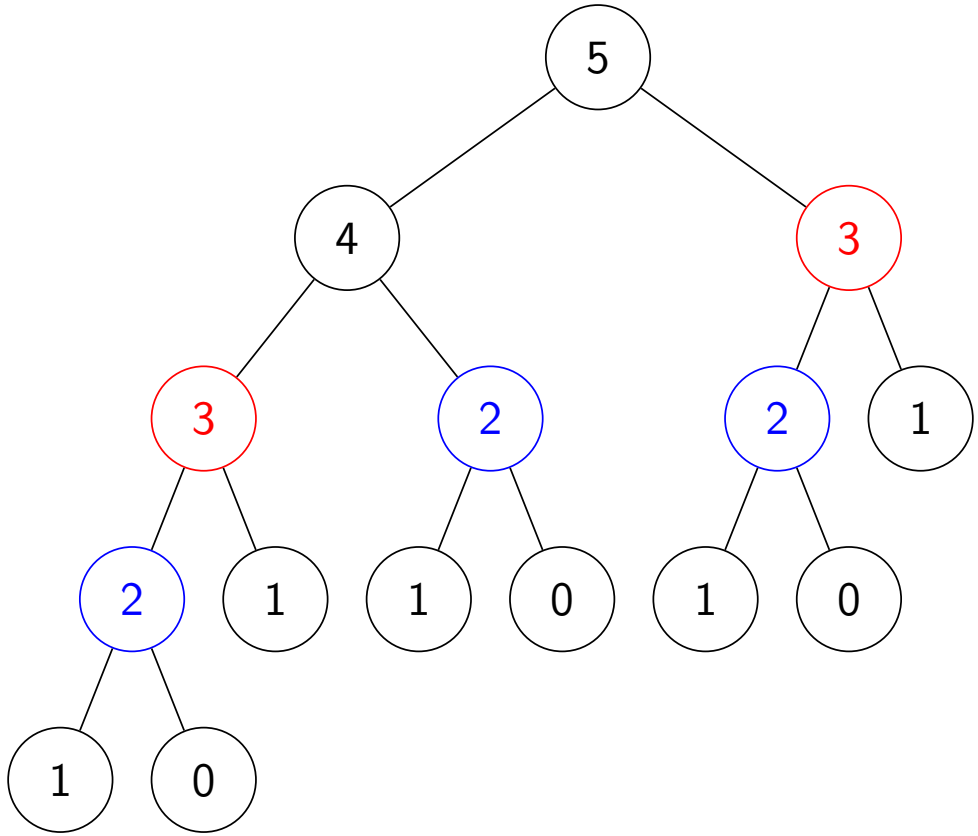
```
1 function FibRec(n)
2     if n ≤ 1 then
3         return n
4     end
5     return FibRec(n − 1) + FibRec(n − 2)
6 end
```

Esegui in Colab: Python · C++ · Java

Esempio (1/2)

- Nella prossima slide: l'albero di ricorsione di `FibRec(5)`
- Ogni nodo è una chiamata, etichettata con il suo parametro
- Il sotto-albero di `FibRec(3)` viene calcolato due volte, la chiamata `FibRec(2)` tre volte

Esempio (2/2)



L'albero ha 15 nodi, mentre servono solo 6 valori diversi $F(0), F(1), \ldots, F(5)$.

Osservazione 4: numero di chiamate di FibRec

Per ogni $n \geq 0$, *il numero di chiamate eseguite da* `FibRec`(n) *è*

$$C(n) = 2F(n+1) - 1.$$

Dimostrazione

Le chiamate soddisfano

$$C(0) = C(1) = 1, \qquad C(n) = 1 + C(n-1) + C(n-2) \text{ per } n \geq 2.$$

Dimostriamo la formula per induzione forte.

Casi base: $C(0) = 1 = 2F(1) - 1$ e $C(1) = 1 = 2F(2) - 1$.

Se la formula vale per $n-1$ e $n-2$, allora

$$\begin{aligned} C(n) &= 1 + \big(2F(n) - 1\big) + \big(2F(n-1) - 1\big) \\ &= 2\big(F(n) + F(n-1)\big) - 1 \\ &= 2F(n+1) - 1. \end{aligned}$$

□

Osservazione 5: crescita dei numeri di Fibonacci

Sia $\varphi = \frac{1+\sqrt{5}}{2}$. *Per ogni* $n \geq 1$,

$$\varphi^{n-2} \leq F(n) \leq \varphi^{n-1}.$$

Quindi $F(n) = \Theta(\varphi^n)$ *e il numero di passi di* `FibRec` *è* $S(n) = \Theta(C(n)) = \Theta(\varphi^n)$.

Dimostrazione (1/2)

Il numero φ soddisfa $\varphi^2 = \varphi + 1$.

Casi base:

$$\varphi^{-1} \approx 0.618 \leq F(1) = 1 \leq \varphi^0 = 1,$$

$$\varphi^0 = 1 \leq F(2) = 1 \leq \varphi \approx 1.618.$$

Per $n \geq 3$, per induzione forte,

$$\begin{aligned} F(n) &= F(n-1) + F(n-2) \\ &\geq \varphi^{n-3} + \varphi^{n-4} = \varphi^{n-4}(\varphi + 1) = \varphi^{n-4}\varphi^2 = \varphi^{n-2}. \end{aligned}$$

Dimostrazione (2/2)

Allo stesso modo

$$F(n) \leq \varphi^{n-2} + \varphi^{n-3} = \varphi^{n-3}(\varphi + 1) = \varphi^{n-1}.$$

Di conseguenza,

$$\varphi^{-2}\varphi^n \leq F(n) \leq \varphi^{-1}\varphi^n, \quad \text{cioè} \quad F(n) = \Theta(\varphi^n).$$

Ogni chiamata esegue un numero costante di passi (oltre alle sue sotto-chiamate), quindi

$$S(n) = \Theta(C(n)) = \Theta(2F(n+1) - 1) = \Theta(\varphi^n).$$

□

Quante chiamate?

- `FibRec(40)` esegue

$$2F(41) - 1 = 331\,160\,281$$

chiamate per calcolare $F(40) = 102\,334\,155$

- `FibRec(50)` esegue

$$2F(51) - 1 = 40\,730\,022\,147$$

chiamate

Memoizzazione

- Lo spreco di `FibRec` si elimina memorizzando in una tabella $memo[0..n]$ i valori già calcolati (**memoizzazione**)
- Prima di calcolare $F(k)$ si verifica se è già nella tabella

Algorithm: FibMemo

Input : Un numero naturale n e una tabella $memo[0..n]$ le cui voci sono tutte *non definite*

Output : Il numero di Fibonacci $F(n)$

```
1 function FibMemo(n, memo)
2     if n ≤ 1 then
3         return n
4     end
5     if memo[n] è non definita then
6         memo[n] ←
            FibMemo(n − 1, memo) + FibMemo(n − 2, memo)
7     end
8     return memo[n]
9 end
```

Esegui in Colab: Python · C++ · Java

Osservazione 6: numero di chiamate di FibMemo

Per ogni $n \geq 2$, `FibMemo`$(n, memo)$ *(con tutte le voci di memo non definite) esegue esattamente* $2n - 1$ *chiamate. Quindi* $S(n) = \Theta(n)$*; la memoria usata è* $\Theta(n)$ *(la tabella e la pila delle chiamate).*

Dimostrazione (1/2)

Sia $M(k)$ il numero di chiamate eseguite da `FibMemo`$(k, memo)$ quando le voci $memo[0..k]$ non sono definite.

- Per $k = 2$ la chiamata ne effettua due, entrambe casi base: $M(2) = 3$
- Per $k \geq 3$ la chiamata effettua la chiamata `FibMemo`$(k-1, memo)$, che esegue $M(k-1)$ chiamate e riempie $memo[2..k-1]$
- Poi effettua la chiamata `FibMemo`$(k-2, memo)$, che trova $memo[k-2]$ già definita (oppure è un caso base se $k-2 \leq 1$) e termina subito

Dimostrazione (2/2)

Quindi

$$M(k) = 1 + M(k-1) + 1 = M(k-1) + 2,$$

e per l'Osservazione 1

$$M(n) = M(2) + 2(n-2) = 2n - 1.$$

- Ogni chiamata esegue un numero costante di passi, quindi $S(n) = \Theta(n)$
- La tabella ha $n+1$ voci e la profondità della ricorsione è n



L'algoritmo iterativo

- Calcolando i valori $F(2), F(3), \ldots, F(n)$ in ordine crescente, ogni valore richiede solo i due precedenti
- Non serve nemmeno la tabella: bastano due variabili

Algorithm: FibIt (1/2)

Input : Un numero naturale n
Output : Il numero di Fibonacci $F(n)$

```
1 function FibIt(n)
2 |   if n ≤ 1 then
3 |   |   return n
4 |   end
5 |   prev := 0                          // F(i − 2)
6 |   curr := 1                          // F(i − 1)
7 |   i := 2
```

Algorithm: FibIt (2/2)

```
function FibIt(n) (continua)
8     while i ≤ n do
9         next := prev + curr
10        prev ← curr
11        curr ← next
12        i ← i + 1
13    end
14    return curr
15 end
```

Esegui in Colab: Python · C++ · Java

Tempo di esecuzione di FibIt

Il ciclo viene eseguito $n - 1$ volte:

$$S(n) = \Theta(n),$$

con $\Theta(1)$ di memoria aggiuntiva.

Tre algoritmi per i numeri di Fibonacci

Algoritmo	tempo	spazio
`FibRec` (ricorsione ingenua)	$\Theta(\varphi^n)$	$\Theta(n)$
`FibMemo` (ricorsione + memoizzazione)	$\Theta(n)$	$\Theta(n)$
`FibIt` (iterazione, bottom-up)	$\Theta(n)$	$\Theta(1)$

- Quando i sottoproblemi si sovrappongono, la tecnica giusta è risolvere ogni sottoproblema **una sola volta** e memorizzarne il valore: top-down con la memoizzazione, oppure bottom-up con un ciclo
- Questa è l'idea della **programmazione dinamica**, che verrà studiata più avanti nel corso

Riepilogo

Sottoproblemi sovrapposti:

- `FibRec`(n) esegue $C(n) = 2F(n+1) - 1$ chiamate: $S(n) = \Theta(\varphi^n)$
- l'albero di ricorsione di `FibRec`(5) ha 15 nodi per 6 valori diversi

Eliminare lo spreco:

- `FibMemo`: $2n - 1$ chiamate, tempo $\Theta(n)$ e memoria $\Theta(n)$
- `FibIt`: tempo $\Theta(n)$ e memoria $\Theta(1)$
- risolvere ogni sottoproblema una sola volta: l'idea della programmazione dinamica

Spazio

Introduzione: Spazio

- Le sezioni precedenti hanno confrontato i numeri di passi
- Passiamo ora alla seconda risorsa, la memoria, che per un algoritmo ricorsivo è governata dalla **profondità della ricorsione**
- Calcoliamo la profondità degli algoritmi ricorsivi del corso, e vediamo come la **ricorsione in coda** si può eliminare meccanicamente, ottenendo un ciclo con memoria aggiuntiva costante

Definizione 1: profondità della ricorsione

*La **profondità della ricorsione** $D(n)$ di un algoritmo ricorsivo su un input con parametro di dimensione n è il massimo numero di chiamate attive contemporaneamente durante l'esecuzione. È il numero di nodi del cammino più lungo dalla radice a una foglia dell'albero di ricorsione, cioè l'altezza dell'albero più uno.*

Profondità e memoria

- Quando le chiamate vengono eseguite una dopo l'altra, sono attive solo le chiamate sul cammino dalla radice alla chiamata corrente
- Quindi, se ogni frame ha dimensione costante, la memoria usata dalla pila delle chiamate è $\Theta(D(n))$
- La profondità **non** è il numero di chiamate: `FibRec`(n) esegue $\Theta(\varphi^n)$ chiamate, ma al più n di esse sono attive contemporaneamente

Esempio (1/2)

La pila delle chiamate della somma per decremento di uno.

- Nel codice del corso la somma per decremento di uno è la funzione C++ `sumRecDecrease`, con gli array indicizzati da 0:

  $\texttt{sumRecDecrease}(a, n) = \texttt{sumRecDecrease}(a, n-1) + a[n-1]$

- Eseguiamo `sumRecDecrease(a, 3)` sull'array

$a[j]$	$j = 0$	$j = 1$	$j = 2$
	4	7	1

- Nella prossima slide: il contenuto della pila delle chiamate (la cima della pila è a destra; `s` sta per `sumRecDecrease` con l'array `a` omesso)

Esempio (2/2)

passo	pila delle chiamate (chiamate attive)	azione
1	`s(3)`	$n = 3 \neq 0$: chiama `s(2)`, poi somma $a[2]$
2	`s(3) s(2)`	$n = 2 \neq 0$: chiama `s(1)`, poi somma $a[1]$
3	`s(3) s(2) s(1)`	$n = 1 \neq 0$: chiama `s(0)`, poi somma $a[0]$
4	`s(3) s(2) s(1) s(0)`	$n = 0$: restituisce 0
5	`s(3) s(2) s(1)`	restituisce $0 + a[0] = 4$
6	`s(3) s(2)`	restituisce $4 + a[1] = 11$
7	`s(3)`	restituisce $11 + a[2] = 12$

- Al passo 4 ci sono $4 = n + 1$ chiamate attive: la profondità della ricorsione è $n + 1$
- Le addizioni vengono eseguite solo **dopo** che si è raggiunto il caso base, mentre la pila si svuota
- Per confronto, la versione iterativa `sumIt(a, 3)` usa solo le due variabili `s` e `i`: $s = 0 \to 4 \to 11 \to 12$

Profondità degli algoritmi ricorsivi del corso

Algoritmo	caso base	profondità $D(n)$	$D(10^6)$
Fattoriale, somma lineare (decr. di uno)	$n = 0$	$n + 1$	1 000 001
Torre di Hanoi con n dischi	$n = 0$	$n + 1$	1 000 001
`FibRec`(n), $n \geq 1$	$n \leq 1$	n	1 000 000
Somma, Merge Sort, sotto-array (dimezzam.)	$l = r$	$\lceil \log_2 n \rceil + 1$	21
Ricerca binaria ricorsiva (caso peggiore)	$l > r$	$\lfloor \log_2 n \rfloor + 2$	21
Potenza veloce x^n, $n \geq 1$	$n = 0$	$\lfloor \log_2 n \rfloor + 2$	21
MCD ricorsivo(a, b)	$b = 0$	$\Theta(\log \min(a, b))$	–

Profondità logaritmica e lineare

- Per gli algoritmi per dimezzamento, il sotto-array più grande al livello t dell'albero di ricorsione ha $\lceil n/2^t \rceil$ elementi, e raggiunge la dimensione 1 per $t = \lceil \log_2 n \rceil$
- Per la potenza veloce, l'esponente assume i valori $n, \lfloor n/2 \rfloor, \ldots, 1, 0$, che sono $\lfloor \log_2 n \rfloor + 2$
- Una profondità $\Theta(\log_2 n)$ non è mai un problema in pratica (21 frame per $n = 10^6$, 31 per $n = 10^9$)
- Una profondità $\Theta(n)$ è un vero rischio di stack overflow per n grande

Ricorsione in coda e sua eliminazione

Definizione 2: ricorsione in coda

*Una chiamata ricorsiva è una **chiamata in coda** (tail call) se è l'ultima operazione eseguita dalla funzione, e il suo risultato viene restituito così com'è, senza ulteriori calcoli. Un algoritmo ricorsivo è **ricorsivo in coda** (tail recursive) se tutte le sue chiamate ricorsive sono chiamate in coda.*

Ricorsivo in coda o no?

- L'MCD ricorsivo (restituisce $\texttt{GCD}(b, a \bmod b)$) e la ricerca binaria ricorsiva (restituisce il risultato della ricerca in una metà) sono ricorsivi in coda
- Il fattoriale ($n \cdot \texttt{Fact}(n-1)$), la somma per decremento di uno ($\texttt{SumDecrease}(a, n-1) + a[n]$) e `FibRec` **non** sono ricorsivi in coda
- Dopo che la chiamata ricorsiva termina resta ancora una moltiplicazione o un'addizione da eseguire, quindi il frame deve essere mantenuto

Una chiamata in coda è un salto

- In una chiamata in coda, il frame della funzione chiamante non serve più: dopo la chiamata non resta nulla da fare
- Quindi la chiamata può essere sostituita da un **aggiornamento dei parametri** seguito da un salto all'inizio della funzione
- Cioè da un ciclo

Eliminazione meccanica della ricorsione in coda

1. Racchiudere il corpo della funzione in un ciclo **while** la cui condizione è la negazione della condizione del caso base.

2. Sostituire ogni chiamata in coda $f(e_1, \ldots, e_k)$ con l'assegnazione **simultanea** dei nuovi valori $e_1, \ldots, e_k$ ai parametri (usare variabili temporanee quando un nuovo valore dipende da un parametro che viene modificato).

3. Spostare il return del caso base dopo il ciclo.

Esempio: MCD ricorsivo ⇒ MCD iterativo (1/2)

Algorithm: GCDRec

```
1 function GCDRec(a, b)
2     if b = 0 then
3         return a
4     end
5     return GCDRec(b, a mod b)
6 end
```

Algorithm: GCDIt

```
1 function GCDIt(a, b)
2     while b ≠ 0 do
3         r := a mod b
4         a ← b
5         b ← r
6     end
7     return a
8 end
```

Esegui in Colab: Python · C++ · Java

Esempio: MCD ricorsivo $\Rightarrow$ MCD iterativo (2/2)

- Condizione di base $b = 0 \Rightarrow$ condizione del ciclo $b \neq 0$
- Chiamata in coda $\texttt{GCDRec}(b, a \bmod b) \Rightarrow$ $(a, b) \leftarrow (b, a \bmod b)$; la variabile temporanea r è necessaria perché $a \bmod b$ deve essere calcolato con il *vecchio* valore di a
- Con $(a, b) = (252, 105)$ entrambi gli algoritmi passano per le coppie

$$(252, 105), \quad (105, 42), \quad (42, 21), \quad (21, 0)$$

e restituiscono 21: quattro chiamate contro tre iterazioni

Esempio: ricerca binaria ricorsiva $\Rightarrow$ iterativa (1/3)

Algorithm: RecBinarySearch

```
1  function RecBS(a, l, r, m)
2      if l > r then
3          return −1
4      end
5      mid := l + ⌊(r − l)/2⌋
6      if a[mid] = m then
7          return mid
8      end
9      if a[mid] < m then
10         return RecBS(a, mid + 1, r, m)
11     else
12         return RecBS(a, l, mid − 1, m)
13     end
14 end
```

Esegui in Colab: Python · C++ · Java

Esempio: ricerca binaria ricorsiva ⇒ iterativa (2/3)

Algorithm: ItBinarySearch

```
1  function ItBS(a, l, r, m)
2      while l ≤ r do
3          mid := l + ⌊(r − l)/2⌋
4          if a[mid] = m then
5              return mid
6          end
7          if a[mid] < m then
8              l ← mid + 1
9          else
10             r ← mid − 1
11         end
12     end
13     return −1
14 end
```

Esegui in Colab: Python · C++ · Java

Esempio: ricerca binaria ricorsiva $\Rightarrow$ iterativa (3/3)

- Condizione di base $l > r \Rightarrow$ condizione del ciclo $l \leq r$
- Il secondo caso base ($a[mid] = m$) resta dentro il ciclo come **return**
- Chiamata in coda $\texttt{RecBS}(a, mid + 1, r, m) \Rightarrow l \leftarrow mid + 1$
- Chiamata in coda $\texttt{RecBS}(a, l, mid - 1, m) \Rightarrow r \leftarrow mid - 1$ (i parametri che non cambiano non vengono assegnati)
- Chiamando $\texttt{ItBS}(a, 1, n, m)$ otteniamo esattamente la `BinarySearch` iterativa del capitolo «Ricerca lineare e binaria»

Osservazione 7: eliminazione della ricorsione in coda

Sia f un algoritmo ricorsivo in coda le cui chiamate eseguono un numero costante di passi (oltre alla chiamata ricorsiva) e usano frame di dimensione costante, e sia g l'algoritmo iterativo ottenuto con l'eliminazione meccanica della ricorsione in coda. Allora g restituisce lo stesso risultato di f, il suo numero di passi è dello stesso ordine di quello di f, e usa $\Theta(1)$ di memoria aggiuntiva invece di $\Theta(D(n))$.

Dimostrazione (1/2)

- Siano $p_1, p_2, \ldots, p_k$ i valori dei parametri nelle successive chiamate di f, dove p_k soddisfa una condizione di base
- La j-esima iterazione del ciclo di g inizia con i parametri uguali a p_j ed esegue le stesse operazioni della chiamata di f con parametri p_j, tranne che la chiamata in coda è sostituita dall'assegnazione di p_{j+1}
- Quindi g passa per gli stessi valori $p_1, \ldots, p_k$ e restituisce, alla fine, ciò che restituisce la chiamata con parametri p_k
- Poiché ogni chiamata di f restituisce così com'è il valore della successiva, questo è anche il risultato di f

Dimostrazione (2/2)

- Le k chiamate di f e le $k-1$ iterazioni di g (più il test finale e il return) eseguono gli stessi passi a meno di una costante per chiamata
- Quindi i due numeri di passi sono dello stesso ordine
- Infine, g usa solo i parametri e un numero costante di variabili temporanee

□

Accumulatori

- Una funzione che non è ricorsiva in coda può talvolta essere resa ricorsiva in coda con un parametro aggiuntivo (un **accumulatore**) che porta il risultato parziale

- Per esempio, $\texttt{FactAcc}(n, acc)$ restituisce
  - acc se $n = 0$,
  - $\texttt{FactAcc}(n - 1, n \cdot acc)$ altrimenti,

  e $n! = \texttt{FactAcc}(n, 1)$

- La sua eliminazione dà il solito ciclo $acc \leftarrow acc \cdot n$, $n \leftarrow n - 1$

Lo fanno i compilatori al posto nostro?

- I compilatori ottimizzanti spesso eliminano automaticamente le chiamate in coda
- Per esempio, g++ 15.2 con l'opzione `-O2` elimina le chiamate ricorsive delle funzioni C++ `gcdRec` e `binarySearchRec` del codice del corso, mentre con `-O0` le mantiene
- Tuttavia, lo standard C++ non lo garantisce, e Python e Java non lo fanno mai
- Per essere sicuri che la memoria sia $\Theta(1)$, si scriva il ciclo

Punti chiave sullo spazio

- La pila delle chiamate usa una memoria proporzionale alla profondità della ricorsione, non al numero di chiamate
- Una profondità logaritmica è innocua; una profondità lineare (decremento di uno) rischia lo stack overflow
- La ricorsione in coda può essere eliminata meccanicamente, ottenendo un ciclo con $\Theta(1)$ di memoria aggiuntiva

Riepilogo

Profondità della ricorsione:

- massimo numero di chiamate attive contemporaneamente; memoria della pila $\Theta(D(n))$
- $\Theta(\log_2 n)$ per dimezzamento (innocua), $\Theta(n)$ per decremento di uno (stack overflow)

Ricorsione in coda:

- la chiamata ricorsiva è l'ultima operazione e il suo risultato viene restituito così com'è
- eliminazione meccanica: `GCDRec` $\Rightarrow$ `GCDIt`, `RecBS` $\Rightarrow$ `ItBS`, con $\Theta(1)$ di memoria aggiuntiva
- accumulatori; i compilatori possono farlo, ma non è garantito

Esperimento

Introduzione: Esperimento

- L'analisi asintotica prevede come cresce il numero di passi con n
- In questa sezione verifichiamo queste previsioni su un calcolatore reale, con il codice C++ del corso
- Tre esperimenti: Insertion Sort contro Merge Sort, ricorsione ingenua contro iterazione per i numeri di Fibonacci, e le tre versioni della somma di un array (tempo e pila)

Il codice del corso

- Tutti gli algoritmi di questo capitolo sono implementati nel codice del corso, nel file `Recursion.cpp` (con `Recursion.h`); gli array sono indicizzati da 0
- Gli esperimenti sono nel file `Experiments.cpp`, e gli algoritmi di ordinamento `insertionSort` e `mergeSort` sono nel file `Sortings.cpp`
- Tre implementazioni della somma di un array:
  - `sumIt`: un ciclo;
  - `sumRecHalving`: somma per dimezzamento, profondità $\Theta(\log n)$;
  - `sumRecDecrease`: somma per decremento di uno, profondità $\Theta(n)$.
- Tre implementazioni dei numeri di Fibonacci: `fibonacciRec`, `fibonacciMemo` (con una tabella di $n+1$ posizioni, tutte inizializzate a -1) e `fibonacciIt`

Impostazione sperimentale (1/2)

- I tempi di esecuzione sono stati misurati l'11 ottobre 2026 su un calcolatore con processore Intel Core i5-14500T (20 CPU logiche, frequenza massima 4.8 GHz), con sistema operativo Ubuntu 26.04 LTS (kernel Linux 7.0)
- Il codice `C++` del corso è stato compilato con `g++` 15.2.0 e l'opzione `-O2`
- La dimensione predefinita della pila è 8 MB (`ulimit -s` $\Rightarrow$ 8192 KB)
- La funzione `runRecursionExperiments` del file `Experiments.cpp` misura il tempo reale di ogni calcolo e lo ripete finché non sono trascorsi almeno 0.2 secondi (e almeno 3 ripetizioni); ogni tempo nelle tabelle è la media sulle ripetizioni

Impostazione sperimentale (2/2)

- **Ordinamento**: array di $n = 10^3$, 10^4 e 10^5 interi casuali tra 1 e 10^6 (seme 12345), ordinati con `insertionSort`, con il `mergeSort` ricorsivo e con `mergeSortBottomUp`
- **Fibonacci**: `fibonacciRec`(n) e `fibonacciIt`(n) per $n = 20, 25, 30, 35, 40$; le chiamate di `fibonacciRec` sono contate da un contatore nel codice; `fibonacciIt` viene ripetuta a blocchi di 10^5 chiamate
- **Somma di un array**: `sumIt`, `sumRecHalving` e `sumRecDecrease` su array di $n = 10^5$, 10^6 e 10^7 interi casuali tra 1 e 100, con il limite della pila rimosso (`ulimit -s unlimited`); un'esecuzione separata verifica se `sumRecDecrease` termina con la pila predefinita di 8 MB

Come leggere i tempi misurati

I tempi misurati dipendono dalla macchina,
dal compilatore e dalle sue opzioni,
dagli altri programmi in esecuzione
e dalla memoria cache.

Non vanno confrontati in termini assoluti:
ciò che conta è **come crescono** con n,
che è previsto dall'analisi asintotica
ed è lo stesso su ogni macchina.

Ordinamento: Insertion Sort contro Merge Sort

n	`insertionSort`	`mergeSort` (ricorsivo)	`mergeSortBottomUp`
10^3	0.0695 ms	0.0361 ms	0.0332 ms
10^4	5.38 ms	0.654 ms	0.629 ms
10^5	525 ms	7.76 ms	7.43 ms

- Da $n = 10^4$ a $n = 10^5$ il tempo dell'Insertion Sort viene moltiplicato per circa 98, vicino al fattore 100 previsto da $\Theta(n^2)$

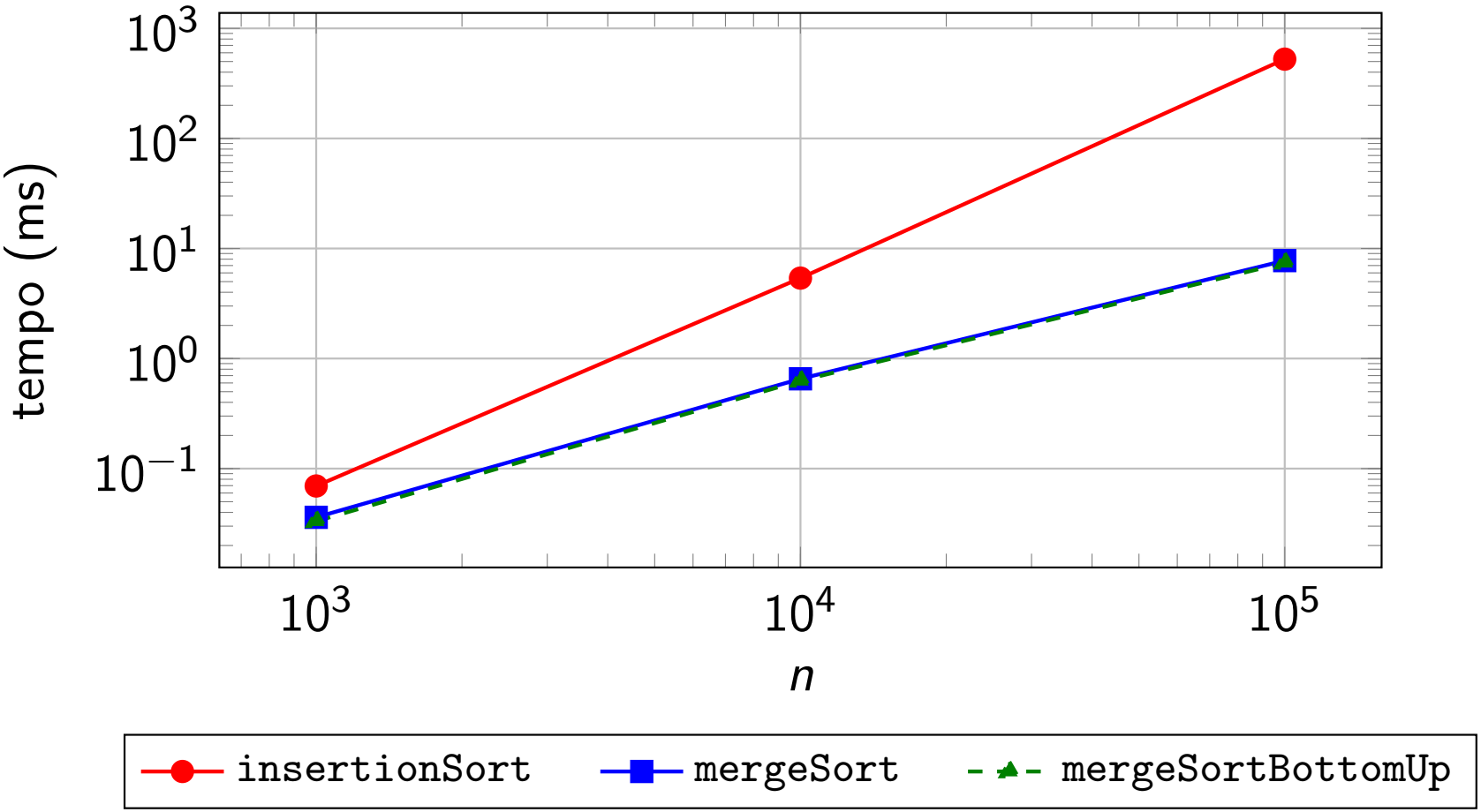
Ordinamento: i fattori di crescita

- Da $n = 10^4$ a $n = 10^5$ il tempo del Merge Sort viene moltiplicato per circa 12, vicino al fattore previsto da $\Theta(n \log_2 n)$:

$$10 \cdot \frac{\log_2 10^5}{\log_2 10^4} = 12.5$$

- Da $n = 10^3$ a $n = 10^4$ i fattori sono più piccoli, circa 77 e 18: per array piccoli i costi costanti e la memoria cache pesano ancora sulla misura
- Per $n = 10^5$ il Merge Sort è circa 70 volte più veloce
- La versione ricorsiva e quella bottom-up hanno praticamente lo stesso tempo: il guadagno è dovuto al divide et impera, non alla ricorsione

Ordinamento: tempi di esecuzione misurati



Scale logaritmiche su entrambi gli assi.

Leggere un grafico log-log

In un grafico con scale logaritmiche su entrambi gli assi
una potenza n^k è una retta di pendenza k.

La retta dell'Insertion Sort ha pendenza vicina a 2,
le rette del Merge Sort
una pendenza leggermente maggiore di 1.

Fibonacci: ricorsione ingenua contro iterazione

n	$F(n)$	chiamate di `fibonacciRec`	`fibonacciRec`	`fibonacciIt`
20	6 765	21 891	0.00808 ms	4.0 ns
25	75 025	242 785	0.0699 ms	4.8 ns
30	832 040	2 692 537	0.764 ms	7.4 ns
35	9 227 465	29 860 703	9.15 ms	8.4 ns
40	102 334 155	331 160 281	93.8 ms	9.5 ns

- Il numero di chiamate è stato contato con un contatore nel codice e coincide con $2F(n+1)-1$ (Osservazione 4)

Fibonacci: i fattori di crescita

- Quando n aumenta di 5, il tempo di `fibonacciRec` viene moltiplicato per un fattore compreso tra circa 9 e 12, in accordo con $\varphi^5 \approx 11.09$
- Il tempo di `fibonacciIt` resta di pochi nanosecondi
- Estrapolando con il fattore φ per ogni unità di n dal tempo misurato per $n = 40$:
  - `fibonacciRec`(50) richiede circa $93.8 \cdot \varphi^{10} \approx 11\,500$ ms, cioè circa 12 secondi;
  - `fibonacciRec`(60) richiede circa $93.8 \cdot \varphi^{20}$ ms, cioè circa 24 minuti;
  - `fibonacciRec`(100) richiede circa $93.8 \cdot \varphi^{60}$ ms, cioè circa 10 000 anni.

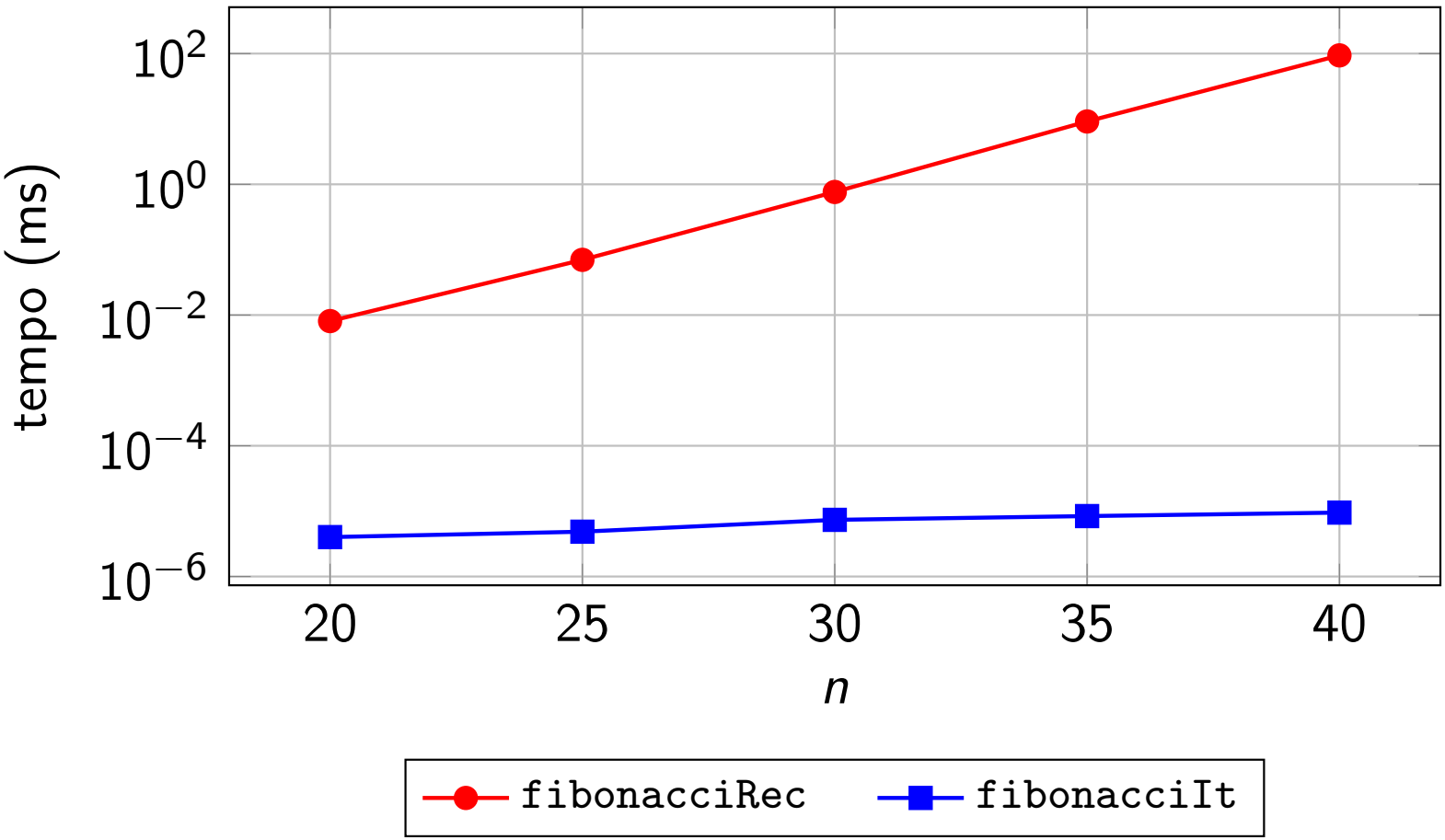
Fibonacci: il tempo per chiamata

- Il compilatore ottimizza il codice: con `-O2` fonde più livelli della ricorsione in una singola chiamata di funzione
- Quindi il tempo per chiamata,

$$93.8 \text{ ms}/331\,160\,281 \approx 0.28 \text{ ns},$$

  è minore di quanto ci si aspetterebbe
- Questo cambia la costante, non la crescita esponenziale

Fibonacci: tempi di esecuzione misurati



Scala logaritmica sull'asse verticale.

Leggere un grafico semi-logaritmico

In un grafico semi-logaritmico
una funzione esponenziale è una retta.

La pendenza della retta rossa è circa

$$\log_{10} \varphi \approx 0.21$$

per ogni unità di n.

Somma di un array: tempo e pila

n	`sumIt`	`sumRecHalving`	`sumRecDecrease`	`sumRecDecrease`, pila 8 MB
10^5	0.0215 ms	0.0960 ms	0.207 ms	OK
10^6	0.210 ms	0.910 ms	2.24 ms	crash
10^7	2.69 ms	9.01 ms	34.9 ms	crash

- I tre algoritmi hanno lo stesso ordine di crescita $\Theta(n)$: quando n viene moltiplicato per 10, tutti i tempi vengono moltiplicati per circa 10 (tra 9.5 e 16; da $n = 10^6$ a $n = 10^7$ l'array non sta più nella memoria cache e i fattori aumentano)

Somma di un array: sovraccarico delle chiamate e crash

- Le versioni ricorsive sono più lente di un fattore costante: questo è il sovraccarico delle chiamate
  - circa 3–4.5 per la versione per dimezzamento;
  - circa 10–13 per la versione per decremento di uno.

- Con la pila predefinita di 8 MB, `sumRecDecrease` va in crash (segmentation fault) per $n = 10^6$ e $n = 10^7$

- I tempi della tabella sono stati misurati rimuovendo il limite con il comando `ulimit -s unlimited`

Somma di un array: dove avviene il crash (1/2)

- Per bisezione su n (con una precisione di 10^3) abbiamo trovato che, con 8 MB, il massimo n che non va in crash è
  - circa $6.97 \cdot 10^5$ con `-O2`,
  - circa $2.62 \cdot 10^5$ senza ottimizzazione (`-O0`).

- Senza ottimizzazione ogni chiamata usa un frame di 32 byte:

$$\frac{8 \cdot 2^{20}}{32} = 262\,144$$

Somma di un array: dove avviene il crash (2/2)

- Con `-O2` il compilatore fonde quattro livelli della ricorsione in una singola chiamata con un frame di 48 byte, quindi

$$\frac{8 \cdot 2^{20}}{48} \cdot 4 \approx 6.99 \cdot 10^5$$

- L'ottimizzazione cambia la costante, non la profondità $\Theta(n)$
- `sumRecHalving` usa solo $\lceil \log_2 n \rceil + 1 \leq 25$ frame per $n \leq 10^7$ e non va mai in crash
- `sumIt` non usa alcun frame

Riepilogo

Ordinamento:

- fattori vicini a 100 ($\Theta(n^2)$) e a 12.5 ($\Theta(n \log_2 n)$); il Merge Sort ricorsivo e quello bottom-up hanno lo stesso tempo

Fibonacci:

- un fattore circa $\varphi^5 \approx 11.09$ ogni 5 unità di n per `fibonacciRec`, pochi nanosecondi per `fibonacciIt`

Somma di un array:

- stesso ordine $\Theta(n)$, ricorsione più lenta di un fattore costante
- con 8 MB di pila, `sumRecDecrease` va in crash oltre circa $7 \cdot 10^5$

Linee guida

Introduzione: Linee guida

- In quest'ultima sezione raccogliamo ciò che abbiamo imparato in poche regole pratiche
- Quando usare la ricorsione, quando evitarla, e che cosa controllare in ogni caso
- Riassumono l'intero capitolo

Usare la ricorsione quando

- il problema si suddivide in pochi sottoproblemi **indipendenti** di dimensione n/b (divide et impera), in modo che il Master theorem dia un tempo di esecuzione migliore di quello degli algoritmi iterativi (Merge Sort, potenza veloce, Karatsuba);
- la descrizione ricorsiva è quella naturale e comunque non è possibile alcun guadagno asintotico (Torre di Hanoi, e più avanti le visite di alberi e grafi);
- la profondità della ricorsione è piccola, tipicamente $\Theta(\log_2 n)$.

Evitare la ricorsione quando

- il problema diminuisce solo da n a $n-1$ con lavoro aggiuntivo costante: la ricorsione è un ciclo sotto mentite spoglie, con lo stesso tempo di esecuzione, un sovraccarico delle chiamate, $\Theta(n)$ di memoria di pila e un rischio di stack overflow (fattoriale, somma lineare, ricerca lineare);

- i sottoproblemi si **sovrappongono**: la ricorsione ingenua ripete gli stessi calcoli e può essere esponenziale (Fibonacci); si usi la memoizzazione o, meglio, un ciclo bottom-up (programmazione dinamica).

In ogni caso

- **Considerare sempre lo spazio**: la pila delle chiamate usa una memoria proporzionale alla profondità della ricorsione. La ricorsione in coda può essere eliminata meccanicamente, ottenendo un ciclo con $\Theta(1)$ di memoria aggiuntiva.

- **Considerare sempre il miglior algoritmo iterativo**: un algoritmo ricorsivo può migliorare la forza bruta ed essere comunque battuto da un ciclo più astuto (l'algoritmo di Kadane per il sotto-array di somma massima).

Riepilogo (1/2)

Ricorsione e iterazione:

- stessa potenza espressiva; la ricorsione paga sovraccarico delle chiamate, memoria di pila e rischio di stack overflow

Divide et impera:

- il guadagno viene da sottoproblemi bilanciati e indipendenti, non dalla ricorsione (`MergeSortBottomUp`, `PowerIt`); il miglior algoritmo può essere iterativo (`Kadane`)

Decremento di uno:

- un ciclo sotto mentite spoglie: stesso tempo, profondità $\Theta(n)$

Riepilogo (2/2)

Sottoproblemi sovrapposti:

- `FibRec` è $\Theta(\varphi^n)$; `FibMemo` e `FibIt` sono $\Theta(n)$

Spazio:

- memoria di pila proporzionale alla profondità; la ricorsione in coda diventa un ciclo con memoria $\Theta(1)$

Esperimento:

- i tempi misurati crescono come previsto; `sumRecDecrease` va in crash con la pila predefinita